

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Error Analysis Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.*
(BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 25

Code of content:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the student: _____

Assessment Tasks Error Analysis Task

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.
2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.
3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.
4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.
5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Error Analysis Task Rubric

Corrective Action	25%	Proposes effective and feasible corrections	Reasonable corrections	Basic corrections	Ineffective corrections
Use of Hadoop / Integration Concepts	15%	Applies relevant concepts appropriately	Mostly appropriate application	Partial application	Incorrect application

Clarity	10%	Clear and direct explanation	Generally clear	Somewhat unclear	Poorly presented
---------	-----	------------------------------	-----------------	------------------	------------------

1. Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.

The problem is mainly caused by **poor data partitioning and an improper number of reducers**. In Hadoop, input data is divided into splits and processed by mapper tasks. If the splits are highly uneven, some mappers receive much more data than others.

Likely design errors:

1. **Uneven input splits:** Some input files or blocks are much larger than others, causing certain mappers to take longer.
2. **Data skew:** A few keys may contain a very large number of records.
3. **Incorrect number of reducers:** Too few reducers can create bottlenecks, while too many may remain idle.
4. **Poor partitioning:** Data may not be distributed evenly among reducers.
5. **Small files problem:** A very large number of small files can create inefficient mapper execution.

Corrective actions:

- Use appropriate **HDFS block size** and input-split configuration.
- Combine small files before processing.
- Use a **custom partitioner** when key distribution is uneven.
- Choose the number of reducers according to the data size and cluster resources.
- Use **combiners** where possible to reduce intermediate data.
- Analyze reducer input distribution and identify data-skewed keys.
- Repartition or preprocess highly skewed data.

Conclusion:

The Hadoop job should be redesigned so that **input data and intermediate keys are distributed evenly**. Proper partitioning, suitable split sizes, and an appropriate number of reducers will improve parallelism and prevent reducer idling.

2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.

HDFS is designed to tolerate individual node failures using **data replication and NameNode high availability**. If data becomes unavailable after only one node fails, there is probably an error in the replication or NameNode configuration.

Probable errors:

1. **Replication factor is too low:** If a block has only one copy, failure of that DataNode makes the block unavailable.
2. **Incorrect replica placement:** Multiple replicas may have been placed on the same physical failure domain.
3. **Insufficient DataNodes:** Too few nodes can limit proper replica distribution.
4. **NameNode single point of failure:** A single NameNode without High Availability can make HDFS metadata unavailable.
5. **Delayed re-replication:** The cluster may not be configured to quickly recreate lost replicas.

6. **Incorrect rack awareness:** Replicas may not be distributed across different racks.

Corrective actions:

- Set an appropriate **replication factor**, commonly 3 for important data.
- Distribute replicas across different DataNodes and racks.
- Configure **NameNode High Availability** with an active and standby NameNode.
- Monitor DataNode and NameNode health.
- Enable automatic re-replication of lost blocks.
- Ensure sufficient DataNodes are available.
- Use rack-aware replica placement.

Conclusion:

The system should use proper replication, rack awareness, automatic recovery, and NameNode High Availability. This ensures that failure of one node does not make important data unavailable.

3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.

Duplicate records occur when the same data is processed or inserted more than once. In an ETL and Apache NiFi pipeline, this can happen because of **retries, incorrect flow design, or lack of duplicate detection**.

Likely causes:

1. **Repeated NiFi processing:** The same file or message may be processed multiple times.
2. **Retry mechanism:** A failed transaction may be retried even though the first attempt was already stored.
3. **No unique identifier:** Records may not have a unique ID for duplicate detection.
4. **Incorrect ETL logic:** The ETL process may insert records repeatedly instead of updating existing records.
5. **Poor processor configuration:** NiFi processors may be configured incorrectly.
6. **Source duplication:** The original data source may itself contain duplicate records.

Recommended fixes:

- Assign a **unique ID** to every incoming record.
- Use database **primary keys or unique constraints**.
- Configure NiFi processors carefully to avoid repeated processing.
- Use proper **success and failure relationships**.
- Implement **idempotent processing**, where processing the same record twice produces the same final result.
- Use deduplication logic in the ETL stage.
- Maintain data provenance and logs to identify where duplication occurs.
- Validate records before loading them into the analytics store.

Conclusion:

The pipeline should use **unique identifiers, deduplication, idempotent processing, and proper error handling**. This prevents retries or repeated ingestion from creating duplicate records.

4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.

YARN manages CPU, memory, and other resources in a Hadoop cluster. Resource starvation occurs when one or more jobs consume most of the available resources, leaving insufficient resources for other users.

Probable scheduling errors:

1. **Improper scheduler configuration:** The scheduler may not be configured for fair resource sharing.
2. **No resource quotas:** Users may consume excessive cluster resources.
3. **Incorrect queue configuration:** All jobs may be submitted to a single queue.
4. **Large resource requests:** Some applications may request excessive memory or CPU.
5. **Lack of user limits:** There may be no limit on the number of applications a user can run.
6. **Poor cluster capacity planning:** Available resources may be insufficient for the workload.

Better resource management approach:

- Use **Capacity Scheduler** or **Fair Scheduler** concepts to share resources between users.
- Create separate queues for different departments or workloads.
- Assign appropriate **capacity and resource limits** to each queue.
- Set maximum resource limits for individual applications.
- Configure user/application limits.
- Give priority to critical jobs where necessary.
- Monitor CPU, memory, queues, and application usage.
- Add more cluster nodes when workload consistently exceeds capacity.

Conclusion:

A properly configured **queue-based scheduling system with resource limits and fair allocation** prevents one user or job from consuming all cluster resources and improves overall cluster utilization.

5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Restoring outdated data indicates that the backup system is **not maintaining recent versions or is using stale backup copies**. Replication and backup serve different purposes, so relying only on replication is also a design mistake.

Probable design flaws:

1. **Old backup copies:** The latest backup may not have been completed before failure.
2. **Infrequent backups:** Backups may be performed only occasionally.
3. **Replication treated as backup:** Replicas can also be affected by accidental deletion or corruption.
4. **No versioning:** Previous versions of files may not be retained.
5. **Incorrect backup scheduling:** Backup jobs may not run according to the required recovery objectives.
6. **No backup verification:** Backups may exist but may not have been tested for successful restoration.
7. **Single backup location:** A failure affecting the primary system and backup location can cause data loss.

Corrective actions:

- Perform backups at appropriate intervals based on business requirements.
- Maintain **multiple backup versions**.
- Use incremental and full backup strategies appropriately.
- Store backups in a **separate failure domain or location**.
- Do not depend only on HDFS replication.
- Implement backup integrity checks.
- Perform regular **restore testing**.
- Define **RPO (Recovery Point Objective)** to determine how much recent data can be lost.
- Define **RTO (Recovery Time Objective)** to determine acceptable recovery time.

Conclusion:

The backup system should maintain **recent, versioned, independently stored, and regularly tested backups**. Replication provides availability, but a proper backup strategy is required for recovery from corruption, accidental deletion, or major failures.